

Review Maker — Full Documentation

Table of Contents

1. Overview
2. Roles & How to Use
 - 2.1 For Developers / Reviewers
 - 2.2 For Scrum Masters
 - 2.3 For Administrators
3. System Architecture
 - Components
 - Data Flow
 - File Structure
4. Technology Stack
5. Setup Guide (Any Host)
 - Prerequisites
 - Step 1: Clone the Repository
 - Step 2: Install Dependencies
 - Step 3: Create Discord Application
 - Step 4: Configure Environment
 - Step 5: Deploy Slash Commands
 - Step 6: Start the System
 - Step 7: First Login
 - Step 8: Set Up Auto-Restart (Production)
 - Step 9: Set Up SSH Tunnel (Remote Access)
 - Step 10: Set Up Watchdog (Optional)
6. Discord Commands Reference
 - General Commands (Everyone)
 - Admin Commands (Admin Only)
 - Priority Levels & Deadlines
 - Review Statuses
 - Roles
7. Dashboard Guide
 - Access
 - Tabs
 - Reviewer Table
 - Creating a Review
 - Deleting a User
 - Disabling a Reviewer
 - GitLab Settings (Admin tab)
8. Debugging & Troubleshooting
 - Log Files
 - Health Check

- Common Issues
- Recovery Procedures
- Testing
- 9. Maintenance Guide
 - Weekly Maintenance
 - Monthly Maintenance
 - Updates
 - Backups
 - Portability (Moving to Another Host)
- 10. Security
 - Password Storage
 - Rate Limiting
 - Access Control
 - Discord Security
 - Network Security
 - Logging

Review Maker — Full Documentation

Version: 2.0.0

Status: Release

Last Updated: 2026-06-12

Table of Contents

- 1. Overview
- 2. Roles & How to Use
 - For Developers / Reviewers
 - For Scrum Masters
 - For Administrators
- 3. System Architecture
- 4. Technology Stack
- 5. Setup Guide (Any Host)
- 6. Discord Commands Reference
- 7. Dashboard Guide
- 8. Debugging & Troubleshooting
- 9. Maintenance Guide
- 10. Security

1. Overview

Review Maker is a review assignment and tracking system that integrates Discord, a web dashboard, and optional GitLab/Google Sheets. It automatically assigns reviewers to merge requests with load balancing, specialty matching, and deadline tracking.

Core capabilities:

- Auto-assign reviewers based on load and specialty
 - Track review lifecycle: pending → in_review → fix_needed → fix_made → approved/rejected
 - Deadline management by priority (Imp=5d, Mid=7d, Low=10d) with overdue highlighting
 - Discord slash commands for all review operations
 - Web dashboard for visual management and admin controls
 - GitLab merge request auto-detection and auto-close on merge
 - Auto-reassign reviews when a reviewer leaves or is removed
 - Crash-resistant with auto-restart loops and watchdog monitoring
 - Rate-limited login/password endpoints (5 attempts / 15 min)
-

2. Roles & How to Use

2.1 For Developers / Reviewers

Your role: reviewer OR senior

First-time setup

1. **Go to the Discord server** and run `/link`
2. Select your name from the dropdown (or click “Register” if you are new)
3. Your Discord account is now linked — you can use all commands

Daily workflow

1. Get your assigned reviews:

`/my-reviews`

This shows all reviews assigned to you with their status.

2. Check what needs attention:

```
/review status
```

Shows all active reviews in the system.

3. View review details:

```
/review details id:REV-42
```

4. Approve a review:

```
/review approve id:REV-42
```

Or click the **Approve** button on the Discord embed notification.

5. Reject a review (with reason):

```
/review reject id:REV-42 comment:Needs unit tests
```

Or click the **Reject** button on the Discord embed and fill in the modal.

6. Add a comment to a review:

```
/review comment id:REV-42 text:LGTM overall, just fix the typo
```

7. After fixing issues (if you are the merger):

```
/review fix-done id:REV-42
```

This resets the reviewers who rejected back to pending status and notifies them.

8. Escalate a stuck review (merger or scrum master only):

```
/review escalate id:REV-42 reason:Blocked on design decision
```

9. Create a new review (any linked user):

```
/review create branch:feature/my-feature type:fullstack  
priority:mid
```

Or with a commit reference:

```
/review create-commit commit:a1b2c3d branch:main type:fullstack  
priority:mid
```

10. My info:

`/my-reviews` – assigned reviews
`/info` – your linked account details

11. Leave the team (if you need to step away):

`/leave`

Your active reviews will be auto-reassigned to other available reviewers.

2.2 For Scrum Masters

Your role: `scrum_master`

Scrum Masters have visibility into the review process and can escalate blocked reviews, but cannot be assigned as reviewers.

Key commands

`/review status` – Overview of all active reviews
`/review details id:42` – Full details including deadline
`/history` – Recent activity timeline

Escalating a blocked review

When a review is stuck in `fix_needed` and the merger is not responding:

1. Check the review details: `/review details id:REV-42`
2. Escalate to senior: `/review escalate id:REV-42 reason:Author unresponsive for 3 days`

The senior reviewer will be notified and can decide to approve or reject the review.

Dashboard access

1. Go to `http://<server-ip>:3000` (or the hosted URL)
2. Log in with your credentials (set by an admin)
3. You see: New Review, Active Reviews, All Reviews, History tabs
4. You **cannot** see: Admin tab, Debug tab, Raw Data tab

Monitoring the pipeline

Use the Active Reviews tab on the dashboard to see: - Which reviews are overdue (highlighted in red) - Reviewer loads - Review status distribution

2.3 For Administrators

Your role: admin

Admins have full control over the system. They can also act as reviewers if they have capacity.

User management

Add a user:

```
/admin add-user name:"John Doe" speciality:Fullstack role:reviewer
```

The system generates a password automatically — use `/admin passwords` to see it.

Remove a user:

```
/admin remove-user user:"John Doe"
```

Active reviews assigned to this user are **auto-reassigned** to available reviewers.

Change a user's role:

```
/admin set-role user:"John Doe" role:senior
```

Only one senior at a time — setting a new senior demotes the old one. Non-review roles (manager, scrum_master) automatically get specialty = None.

Change a user's load manually:

```
/admin set-load user:"John Doe" load:2
```

Set weekly cap:

```
/admin set-weekly user:"John Doe" cap:5
```

Password management

View all passwords:

```
/admin passwords
```

Reset a password (generates new one):

```
/admin reset-password user:"John Doe"
```

Set a specific password:

```
/admin set-password user:"John Doe" password:MySecurePass123
```

Discord management

Unlink a Discord account:

```
/admin unlink user:"John Doe"
```

The user will need to run `/link` again.

System settings

View current settings:

```
/admin settings
```

Update a setting:

```
/admin settings key:maxLoad value:5
```

Key settings:	Key	Description	Default					
	maxLoad	Global max concurrent reviews per reviewer	3					
	reviewersPerRequest	How many reviewers per review	3					
	nextReviewNumber	Next review ID number	auto					

Dashboard admin panel

1. Go to `http://<server-ip>:3000`
2. Log in with admin credentials
3. You see all tabs including **Admin**, **Raw Data**, **Debug**
4. Admin tab lets you:
 - Edit reviewer settings (role, specialty, load, limits, disabled)
 - Manage passwords
 - Configure GitLab (URL, Token, Project)
 - Sync with Google Sheets

- Delete users (with auto-reassign)
- Repair reviewer loads (recalculate from active reviews)

Deleting a user from the dashboard: 1. Go to Admin tab 2. Find the user in the user management section 3. Click the red “Delete” button 4. Confirm in the dialog 5. Active reviews are reassigned automatically

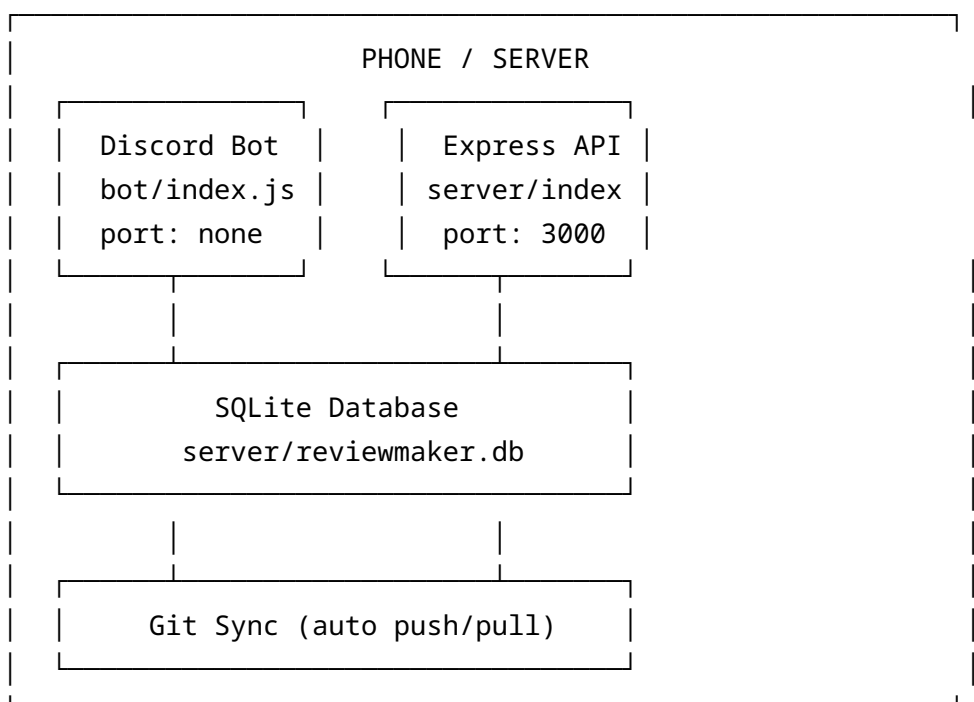
Disabling a reviewer (without deleting): 1. Go to Admin tab 2. Toggle “Disabled” for the reviewer 3. They will no longer be selected for new reviews 4. Existing reviews are NOT affected

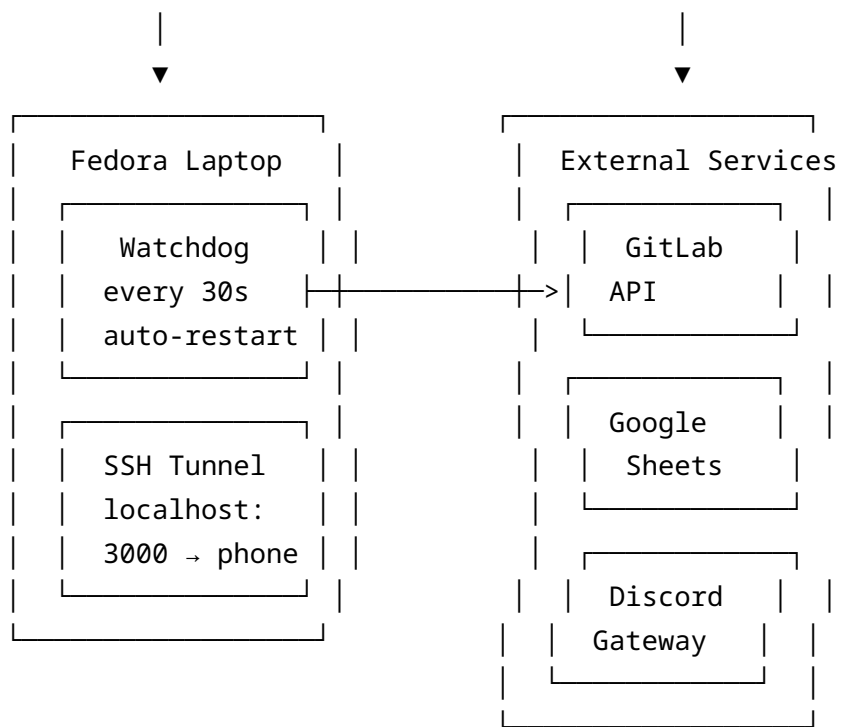
GitLab configuration

1. Go to Admin tab on the dashboard
 2. Scroll to “GitLab Settings”
 3. Fill in:
 - **GitLab URL:** `https://gitlab.example.com`
 - **GitLab Token:** Your personal access token with `read_api` scope
 - **GitLab Project:** Project path (e.g. `group/project`)
 4. Click “Save” — the bot will start syncing MRs and auto-closing reviews on merge
-

3. System Architecture

Components





Data Flow

- Review Creation:** Discord command or Dashboard → API → `selectReviewers()` → Load balancing with specialty matching → Save to SQLite → Discord notification to assignees
- Approval/Rejection:** Discord button or Dashboard → API → Status update → Load decrement → Discord notification to merger
- GitLab Sync:** Every 5 minutes → Fetch open/merged MRs → Update Google Sheet → Auto-close reviews for merged MRs
- Auto-deploy:** Bot auto-pulls from Git every 5 minutes → If changes detected → `npm install` → Restart

File Structure

```

review-maker/
├── bot/
│   ├── index.js           # Discord client, event handlers,
│   │   │                 button/modal logic
│   ├── deploy-commands.js # Slash command registration (guild-
│   │   │                 only)
│   ├── commands/
│   │   ├── admin.js       # Admin subcommands (14 subcommands)
│   │   ├── health.js      # /health - system health check
│   │   └── history.js      # /history - audit log and review
│   └── history
│       └── info.js         # /my-reviews - personal review list
  
```

```

|   |   | — leave.js           # /leave – leave team with auto-
reassign
|   |   | — link.js           # /link – Discord account linking +
register
|   |   | — needattention.js  # /needattention – flag approved
reviews
|   |   | — review.js        # /review – 12 subcommands (create,
approve, etc.)
|   |   | — test.js          # /test – diagnostic command
|   |   | — utils/
|   |   | — data.js          # Load/save, role helpers, reviewer
lookup
|   |   | — embeds.js        # Discord embed builders
|   |   | — reviews.js      # Review logic: create, approve,
disapprove, selectReviewers
| — server/
|   | — index.js             # Express API (30+ endpoints), rate
limiting, auth
|   | — db.js                # SQLite database layer with JSON
migration
|   | — data.json            # Legacy JSON data file (migrated to
SQLite)
|   | — gitlab-sync.js       # GitLab MR polling + auto-close
|   | — discord-sync.js      # Discord approval sync
|   | — sheets-sync.js       # Google Sheets sync
|   | — season-groups.js     # Season group management
|   | — debug-routes.js      # Debug API endpoints
|   | — reviewmaker.db       # SQLite database (auto-created)
| — public/
|   | — login.html           # Login page
|   | — dashboard.html       # Main dashboard (9 tabs)
|   | — css/style.css        # Dashboard styles
|   | — js/
|   |   | — api.js           # REST API client
|   |   | — dashboard.js     # Dashboard UI logic
| — scripts/
|   | — bot-service.sh       # Linux service wrapper
|   | — heartbeat.sh         # Heartbeat monitor
|   | — mr-updater.gs        # Google Apps Script for MR updates
| — tests/
|   | — api.test.js          # Jest API tests
| — logs/                    # Crash log directory
| — .env                     # Environment configuration

```

4. Technology Stack

Component	Technology	Version
Runtime	Node.js	18+
Web Framework	Express	4.18
Discord Library	discord.js	14.26
Database	SQLite (via sql.js)	1.14
Password Hashing	bcryptjs	3.0
Rate Limiting	express-rate-limit	7.1
ORM/Query	sql.js (raw SQL)	—
Frontend	Vanilla HTML/CSS/JS	—
Git Integration	simple-git	3.36
Testing	Jest + Supertest	30.x / 7.x
Google APIs	googleapis	172
CSV Parsing	csv-parser	3.0
Unique IDs	uuid	9.0
Process Manager	tmux (auto-restart loop)	—
Remote Monitor	Custom Bash watchdog	—

5. Setup Guide (Any Host)

This guide covers setting up Review Maker on **any Linux server, VPS, Raspberry Pi, or Android phone (Termux)**.

Prerequisites

- Node.js v18 or higher (check: `node --version`)
- Git (check: `git --version`)
- A Discord Bot Token and Client ID (see below)
- A Discord server where you have “Manage Server” permissions

Step 1: Clone the Repository

```
git clone https://github.com/Artur-Nayman/Review-Maker.git review-maker
cd review-maker
git checkout dashboard-remote
```

Step 2: Install Dependencies

```
npm install
```

Step 3: Create Discord Application

1. Go to Discord Developer Portal
2. Click **New Application** → name it “Review Maker”
3. Go to **Bot** → **Reset Token** → copy the token
4. Enable these **Privileged Gateway Intents**:
 - Server Members Intent
 - Message Content Intent
5. Go to **OAuth2** → copy the **Client ID**
6. Get your **Guild ID**: Discord → User Settings → Advanced → Developer Mode ON → right-click server → Copy ID
7. Invite the bot: open this URL (replace CLIENT_ID):

```
https://discord.com/oauth2/authorize?
client_id=CLIENT_ID&permissions=8&scope=bot%20applications.comm;
```

Step 4: Configure Environment

```
cp .env.example .env
```

Edit .env:

```
# Required
DISCORD_BOT_TOKEN=your_token_here
DISCORD_CLIENT_ID=your_client_id_here
DISCORD_GUILD_ID=your_guild_id_here
```

```
# Server
PORT=3000
```

Step 5: Deploy Slash Commands

```
npm run bot:deploy
```

Commands appear instantly in your Discord server.

Step 6: Start the System

```
# Option A: Run both server and bot together
```

```
npm run all
```

```
# Option B: Run separately (two terminals)
```

```
npm start          # Web dashboard on port 3000
```

```
npm run bot        # Discord bot
```

```
# Option C: Using tmux (recommended for servers)
```

```
tmux new-session -d -s review-bot -n bot 'while true; do node bot/
  index.js; sleep 5; done'
```

```
tmux new-window -t review-bot -n server 'while true; do node
  server/index.js; sleep 5; done'
```

Step 7: First Login

1. Open `http://<your-server-ip>:3000` in a browser
2. Create your first admin user (the system creates a default if none exists)
3. Start adding reviewers and creating reviews

Step 8: Set Up Auto-Restart (Production)

Using the included tmux script:

```
chmod +x scripts/bot-service.sh
./scripts/bot-service.sh
```

Using systemd (Linux servers):

Create `/etc/systemd/system/review-maker.service`:

```
[Unit]
```

```
Description=Review Maker
```

```
After=network.target
```

[Service]

```
Type=simple
```

```
User=youruser
```

```
WorkingDirectory=/path/to/review-maker
```

```
ExecStart=/usr/bin/node server/index.js
```

```
Restart=always
```

```
RestartSec=5
```

```
Environment=NODE_ENV=production
```

[Install]

```
WantedBy=multi-user.target
```

Do the same for the bot (different port not needed, just different ExecStart).

Using the included boot script (Termux Android):

Place in ~/.termux/boot/startup:

```
#!/data/data/com.termux/files/usr/bin/bash
termux-wake-lock
sshd
sleep 3
cd ~/review-maker && bash start_bot.sh
```

Step 9: Set Up SSH Tunnel (Remote Access)

On your laptop or desktop:

```
ssh -L 3000:localhost:3000 -N -p 8022 user@host-ip &
```

Or use the included watchdog:

```
# On Fedora laptop
./watchdog.sh &
```

This monitors the health endpoint, maintains the tunnel, and auto-restarts on failures.

Step 10: Set Up Watchdog (Optional)

The watchdog script at `review-maker-connect/watchdog.sh`: - Checks `http://localhost:3000/api/health` every 30 seconds - After 3 consecutive failures → SSH into host → `git pull` → restart bot - Maintains the SSH tunnel automatically - Logs to `/tmp/review-maker-watchdog.log`

systemd user service:

```
mkdir -p ~/.config/systemd/user
```

Create `~/.config/systemd/user/review-maker-watchdog.service`:

[Unit]

Description=Review Maker Bot Watchdog

[Service]

Type=simple

ExecStart=/home/user/review-maker-connect/watchdog.sh

Restart=always

RestartSec=10

[Install]

WantedBy=default.target

Enable:

```
systemctl --user daemon-reload
```

```
systemctl --user enable review-maker-watchdog.service
```

```
systemctl --user start review-maker-watchdog.service
```

6. Discord Commands Reference

General Commands (Everyone)

Command	Description	Parameters
<code>/link</code>	Link Discord to reviewer account	Select from dropdown or click “Register”
<code>/my-reviews</code>	Show your assigned reviews	None
<code>/review status</code>	Show all active reviews	None

Command	Description	Parameters
/review details	Show review details	id (REV-42 or 42)
/review create	Create review, auto-assign reviewers	branch, type, priority
/review create-commit	Create review from commit	commit, branch, type, priority
/review create-manual	Pick reviewers manually	branch, type, priority
/review approve	Approve a review	id
/review reject	Reject with comment	id, comment
/review fix-done	Mark fixes completed	id
/review escalate	Escalate to senior	id, reason
/review comment	Add comment to review	id, text
/review needattention	Flag approved review	id, comment
/history	Recent system activity	Optional limit
/health	System health status	None

Admin Commands (Admin Only)

Command	Description	Parameters
/admin add-user	Add new user	name, speciality, role
/admin remove-user	Remove user + auto-reassign	user
/admin set-role	Change user role	user, role
/admin set-load	Set reviewer load	user, load
/admin set-weekly	Set weekly cap	user, cap
/admin reviewers	List all reviewers	None
/admin workload	Show loads and caps	None
/admin dashboard	Full system summary	None
/admin passwords	View all passwords	None
/admin reset-password	Reset password	user
	Set specific password	user, password

Command	Description	Parameters
/admin set-password		
/admin settings	View/edit settings	Optional key, value
/admin unlink	Remove Discord link	user
/admin delete-review	Force delete review	id

Priority Levels & Deadlines

Priority	Deadline	Meaning
imp	5 days	Important — high priority
mid	7 days	Medium priority (default)
low	10 days	Low priority

Review Statuses

Status	Meaning
in_review	Review is active, awaiting approvals
fix_needed	One or more reviewers rejected
fix_made	Fixes completed, awaiting re-review
escalated	Escalated to senior reviewer
approved	All required approvals received
rejected	Rejected by senior after escalation

Roles

Role	Can Review?	Dashboard Access	Key Permissions
reviewer	Yes	Limited	Approve/reject, create reviews
senior	Yes	Limited	Escalation decisions
admin	Yes	Full	All admin commands, all tabs
manager	No	Read/Write	Manage reviews, no admin panel
scrum_master	No	Read-only	

Role	Can Review?	Dashboard Access	Key Permissions
			View reviews, can escalate

7. Dashboard Guide

Access

URL: `http://<server-ip>:3000`

Default admin: First user created with role admin.

Tabs

Tab	Visible To	Purpose
New Review	All	Create a new review with auto-assign or manual pick
Active Reviews	All	Live view of all in-progress reviews with deadlines
My Reviews	All	Reviews assigned to you
All Reviews	All	Full list with status filter
Reviewers	All	Reviewer list with load dots, specialty, role
History	All	Completed reviews (last 30 days)
Admin	Admin only	User management, settings, GitLab config
Raw Data	Admin only	Full JSON data dump
Debug	Admin only	API endpoint testing

Reviewer Table

Shows each reviewer with: - **Name** and **Role** badge - **Load dots**: green/red dots showing current load vs capacity - **Specialty**: Fullstack, Frontend, Backend, or None - **Load**: numeric value

Creating a Review

1. Go to **New Review** tab
2. Fill in: Branch name, Review type, Priority, Merger
3. Optionally enter a commit reference
4. Click “Create Review”
5. Reviewers are auto-assigned and notified in Discord

Deleting a User

1. Go to **Admin** tab
2. Find the user in the user list
3. Click **Delete** (red button)
4. Confirm in the dialog
5. Active reviews are auto-reassigned

Disabling a Reviewer

1. Go to **Admin** tab
2. Find the reviewer in the user list
3. Check the **Disabled** checkbox
4. Click Save
5. Reviewer will no longer be picked for new reviews

GitLab Settings (Admin tab)

Configure these in the Admin tab to enable GitLab sync: - **GitLab URL**: Your GitLab instance URL - **GitLab Token**: Personal access token with read_api scope - **GitLab Project**: Namespace/project path (e.g. mygroup/myproject)

Once configured, the bot will: 1. Fetch open MRs every 5 minutes 2. Auto-close reviews when the linked MR is merged 3. Sync new MRs to Google Sheets (if configured)

8. Debugging & Troubleshooting

Log Files

Location	What It Logs
logs/crash.log	Bot unhandled exceptions/rejections
logs/crash-server.log	Server unhandled exceptions/rejections
Server stdout	API requests, data changes, GitLab sync
Bot stdout	Command executions, auto-pull, health checks
/tmp/review-maker-watchdog.log (on Fedora)	Watchdog health checks and restarts

Health Check

GET /api/health

Returns:

```
{
  "status": "ok",
  "db": "ok",
  "uptime": 795.48,
  "timestamp": "2026-06-11T21:21:11.501Z"
}
```

Common Issues

“No available reviewers” when creating a review

- All reviewers are at max load → Check loads with `/admin workload`
- All reviewers are disabled → Check Admin tab for disabled flag
- Only one person in the system → Add more reviewers

Bot does not respond to commands

- Bot not online → Check tmux: `tmux attach -t review-bot`
- Commands not deployed → Run `npm run bot:deploy`
- Bot token expired → Regenerate in Discord Developer Portal

Dashboard shows “N/A” for reviewer load

- The reviewer has role admin, manager, or scrum_master
- Previously: Admins were excluded from reviewable roles
- Now: Admins ARE reviewable (since commit c2a2edc)

Reviews not auto-closing on GitLab merge

- Check GitLab settings in Admin tab
- Ensure token has read_api scope
- Check bot logs for [GitLabSync] messages
- Review branch name must match exactly

Database errors

- Run `npm start` — the DB auto-initializes
- If corrupt: delete `server/reviewmaker.db` and restart (data is also in `server/data.json` for migration)

“Already up to date” but changes are not applied

- Push was not done: `git push origin dashboard-remote`
- Check git log: `git log --oneline -5`

Crash loop (bot keeps restarting)

```
tmux attach -t review-bot
# See the error, fix it, then Ctrl+B D to detach
```

Recovery Procedures

Recover from total crash:

```
# On the phone/host
cd ~/review-maker
bash start_bot.sh
```

```
# Check it's running
tmux list-sessions
```

Repair reviewer loads (if they get out of sync):

```
curl -X POST http://localhost:3000/api/admin/repair-loads
```

Or use the Admin tab → Repair Loads button.

Reset admin password:

```
# Direct SQLite manipulation (on the host)
node -e "
const db = require('./server/db');
db.init().then(() => {
  db.execute(`UPDATE reviewers SET password = '\$2b\$10\$...'
    WHERE role = 'admin'`);
  db.persist();
  console.log('Done');
});
"
```

Testing

npm test

Runs Jest test suite against tests/api.test.js.

9. Maintenance Guide

Weekly Maintenance

1. **Check crash logs:** `tail -20 logs/crash.log logs/crash-server.log`
2. **Verify tmux sessions:** `tmux list-sessions` — should show review-bot
3. **Check watchdog:** `systemctl --user status review-maker-watchdog.service`
4. **Review loads:** /admin workload in Discord
5. **Repair loads if needed:** Admin tab → Repair Loads

Monthly Maintenance

1. **Update dependencies:** `npm update`
2. **Check disk space on host:** `df -h`
3. **Restart bot/server:** `tmux kill-session -t review-bot && bash start_bot.sh`
4. **Verify Git sync:** `git log --oneline -10 server/db.js`

Updates

To update the bot:

```
# On the host
cd ~/review-maker
git pull origin dashboard-remote
npm install
tmux kill-session -t review-bot
bash start_bot.sh
```

The bot also auto-pulls every 5 minutes and restarts itself if changes are detected.

Backups

The SQLite database (`server/reviewmaker.db`) contains all data. Backup regularly:

```
cp server/reviewmaker.db server/backup-$(date +%Y%m%d).db
```

Portability (Moving to Another Host)

1. Install Node.js 18+ and Git
 2. Clone the repo
 3. Copy the `.env` file
 4. Copy `server/reviewmaker.db`
 5. Run `npm install`
 6. Run `npm run bot:deploy` to register commands
 7. Invite the bot to your Discord server
 8. Start with `bash start_bot.sh`
-

10. Security

Password Storage

- All passwords are hashed with **bcrypt** (10 salt rounds)
- Plaintext passwords are migrated to bcrypt on startup
- Administrators can generate or reset passwords via the dashboard or Discord

Rate Limiting

- **Login endpoint:** 5 attempts per 15 minutes per IP
- **Password operations:** 10 attempts per 15 minutes per IP
- Both return clear error messages when rate-limited

Access Control

Role	Assign Reviews	Manage Users	View Passwords	Delete Data
reviewer	✓	✗	✗	✗
senior	✓	✗	✗	✗
admin	✓	✓	✓	✓
manager	✗	✗	✗	✗ (can delete reviews)
scrum_master	✗	✗	✗	✗

Discord Security

- Only linked Discord accounts can use commands
- `/link` requires selecting your name from the list (no impersonation without the admin)
- Admins can unlink any account at any time
- Commands are deployed to a **specific guild only** (not global)

Network Security

- Dashboard served over HTTP by default — use a reverse proxy (nginx/Caddy) for HTTPS
- GitLab token stored in database (settings table)
- Watchdog uses SSH key authentication (no passwords)
- Bot token stored in `.env` file (not in the database)

Logging

- All data changes are logged to the audit log with timestamps
 - Unhandled exceptions are logged to `logs/crash*.log`
 - Every review action (create, approve, reject, delete) is logged with who did it
-

